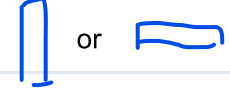


6장. CSS 레이아웃: Flexbox와 Grid

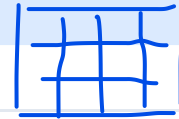
예. 메뉴



block
inline

1차원 형태를 주로 다룬다.

학습 목표 - display 값에 따른 요소의 배치 흐름을 설명할 수 있습니다 - Flexbox의 주축·교차축 개념으로 1차원 정렬을 구현할 수 있습니다 - Grid로 2차원 격자 레이아웃을 설계할 수 있습니다 - position과 z-index로 요소의 위치와 겹침을 제어할 수 있습니다



6.1 display와 흐름 이해하기

도입

브라우저가 가진
기본 흐름

웹 페이지의 요소들은 기본적으로 정해진 규칙에 따라 차례로 놓입니다. 이 규칙을 **정상 흐름 (Normal Flow)**이라고 합니다. 레이아웃을 자유롭게 제어하려면 먼저 이 기본 배치 방식을 이해해야 합니다. display 속성은 각 요소가 얼마나 자리를 차지하고, 옆에 다른 요소를 허용하는지를 결정하는 출발점입니다.

핵심 개념 설명

display 속성 (display property)

display 속성은 요소를 어떤 방식으로 화면에 배치할지 결정합니다. 이 값에 따라 **요소가 줄 전체를 차지할 수도 있고, 다른 요소와 한 줄에 나란히 놓일 수도** 있습니다.

block

inline

주요 값은 세 가지입니다.

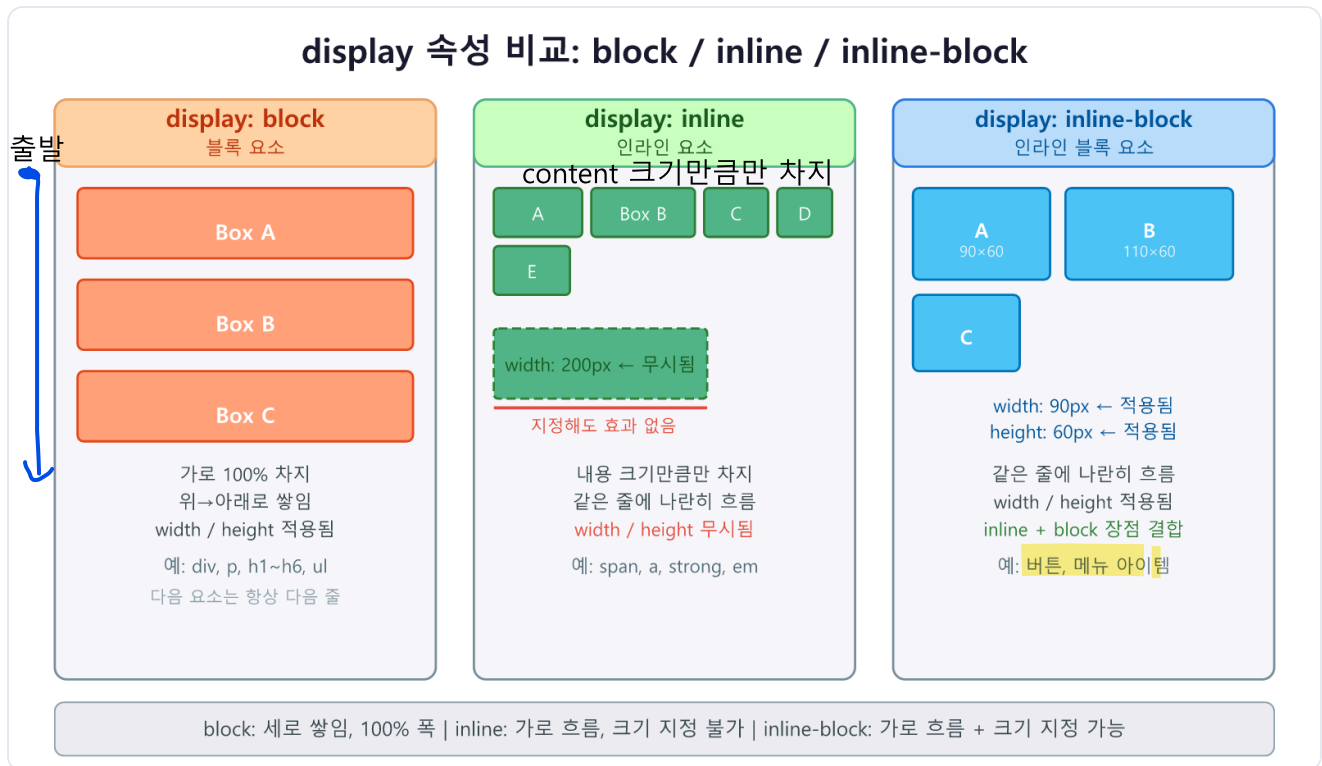
블록(block): 요소가 가로 전체를 차지하고 다음 요소는 아래에 쌓입니다. div, p, h1~h6, ul, li 등이 기본적으로 블록 요소입니다.

인라인(inline): 요소가 내용(content) 크기만큼만 가로를 차지하며 같은 줄에 나란히 흐릅니다. span, a, strong, em 등이 기본적으로 인라인 요소입니다. 인라인 요소는 width와 height가 적용되지 않습니다.

인라인 블록(inline-block): 같은 줄에 나란히 흐르되, `width` 와 `height` 를 지정할 수 있습니다. 인라인의 흐름과 블록의 크기 조절을 동시에 원할 때 사용합니다.

정상 흐름(Normal Flow)

브라우저가 `position` 이나 `float` 같은 특별한 설정 없이 요소를 배치하는 기본 방식을 정상 흐름이라고 합니다. 블록 요소는 위에서 아래로, 인라인 요소는 왼쪽에서 오른쪽으로 흐르는 규칙입니다.



display 속성에 따른 요소 배치 방식 비교 (block / inline / inline-block)

코드로 확인하기

다음 코드는 세 가지 `display` 값의 차이를 한눈에 비교합니다.

```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>display 비교</title>
  <style>
    /* 공통 스타일 */
    .box {
      background-color: #a8d8ea;
      border: 2px solid #2c7be5;
      padding: 8px 16px;
      margin: 4px;
    }

    /* 블록: 줄 전체를 차지 */
    .block-box {
      display: block;
      background-color: #ffd6a5;
      border-color: #f4a261;
    }

    /* 인라인: 내용 크기만큼만, width/height 무시 */
    .inline-box {
      display: inline;
      background-color: #caffbf;
      border-color: #52b788;
      width: 200px; /* 인라인에서는 무시됨 */
      height: 60px; /* 인라인에서는 무시됨 */
    }

    /* 인라인 블록: 한 줄에 나란히 + 크기 지정 가능 */
    .inline-block-box {
      display: inline-block;
      background-color: #bde0fe;
      border-color: #2c7be5;
      width: 120px;
      height: 60px;
      vertical-align: middle;
    }

    .section-title {
      font-weight: bold;
      margin-top: 16px;
      color: #333;
    }
  </style>
</head>
<body>
  <p class="section-title">블록(block): 줄 전체를 차지합니다</p>
  <div class="box block-box">블록 요소 A</div>

```

```

<div class="box block-box">블록 요소 B</div>
<div class="box block-box">블록 요소 C</div>

<p class="section-title">인라인(inline): 같은 줄에 흐릅니다</p>
<span class="box inline-box">인라인 A</span>
<span class="box inline-box">인라인 B</span>
<span class="box inline-box">인라인 C</span>

<p class="section-title">인라인 블록(inline-block): 한 줄 + 크기 지정</p>
<div class="box inline-block-box">IB 요소 A</div>
<div class="box inline-block-box">IB 요소 B</div>
<div class="box inline-block-box">IB 요소 C</div>
</body>
</html>

```

브라우저 렌더링 결과:

[블록(block): 줄 전체를 차지합니다]

```

[———— 블록 요소 A ————]
[———— 블록 요소 B ————]
[———— 블록 요소 C ————]

```

[인라인(inline): 같은 줄에 흐릅니다]

[인라인 A] [인라인 B] [인라인 C]

[인라인 블록(inline-block): 한 줄 + 크기 지정]

```

[ IB 요소 A ] [ IB 요소 B ] [ IB 요소 C ]
(각 박스 120×60px, 한 줄에 나란히)

```

줄별 코드 설명

줄	코드	설명
9	<code>display: block;</code>	블록 배치. 가로 전체를 차지하며 세로로 쌓입니다
16	<code>display: inline;</code>	인라인 배치. 내용 크기만큼만 차지하며 같은 줄에 흐릅니다
18-19	<code>width: 200px; height: 60px;</code>	인라인 요소에서는 <code>width</code> · <code>height</code> 가 적용되지 않습니다
24	<code>display: inline-block;</code>	인라인처럼 한 줄에 흐르면서 <code>width</code> · <code>height</code> 를 지정할 수 있습니다
26-27	<code>width: 120px; height: 60px;</code>	인라인 블록에서는 크기 지정이 정상 적용됩니다

주의: 인라인 요소(`display: inline`)에 `width`, `height` 를 지정해도 적용되지 않습니다. 크기를 지정해야 한다면 `display: inline-block` 으로 변경하세요.

float — 역사적 맥락으로만 알아 두기

CSS3 이전에는 `float` 속성을 이용해 다단 레이아웃을 만들었습니다. 예를 들어 두 개의 `div` 를 `float: left` 로 설정해 나란히 배치하는 방식이었습니다. 그러나 `float` 는 요소를 정상 흐름에서 이탈시켜 부모 높이 계산이 틀어지는 등 여러 부작용을 일으켰습니다.

`clearfix` 같은 해결책이 필요했고 코드가 복잡해졌습니다.

현재는 **Flexbox**와 **Grid**가 표준으로 자리 잡았으므로, `float` 는 텍스트를 이미지 주위로 감싸는 원래 용도에만 사용하고 레이아웃 목적으로는 사용하지 않습니다.

베스트 프랙티스: 레이아웃에는 Flexbox(1차원)와 Grid(2차원)를 사용합니다. `float` 는 이미지 주변 텍스트 흐름에만 제한적으로 사용합니다.

절 요약

- `display: block` → 가로 전체를 차지, 세로로 쌓임
- `display: inline` → 내용 크기만큼만 차지, 같은 줄에 흐름, `width` / `height` 적용 안 됨
- `display: inline-block` → 한 줄에 흐름 + `width` / `height` 적용 가능

- `float` 는 레이아웃 목적으로 사용하지 않음

6.2 Flexbox로 1차원 정렬하기

도입

버튼 여러 개를 가운데 정렬하거나, 카드들을 균일한 간격으로 나란히 배치하고 싶을 때, 과거에는 여러 방법을 조합해야 했습니다. **Flexbox(플렉스박스)**는 이런 1차원(한 방향) 정렬 문제를 직관적으로 해결합니다. "선반 위에 물건을 정렬하는 규칙"처럼, 한 방향으로 아이টে을 유연하게 배치합니다.

핵심 개념 설명

Flexbox의 구조: 컨테이너와 아이টে

Flexbox는 플렉스 컨테이너(**flex container**)와 플렉스 아이টে(**flex item**)의 두 역할로 나뉩니다.



`display: flex` 를 선언한 부모 요소가 컨테이너가 되고, 그 직속 자식 요소들이 자동으로 아이টে이 됩니다. 컨테이너에는 배치 규칙을 지정하고, 아이টে에는 크기 분배를 지정합니다.

주축(main axis)과 교차축(cross axis)

Flexbox의 핵심은 두 축입니다.

- 주축(main axis): 아이টে이 나열되는 방향
- 교차축(cross axis): 주축에 수직인 방향

`flex-direction` 속성으로 주축 방향을 결정합니다.

flex-direction 값	주축 방향	교차축 방향
<code>row</code> (기본값)	왼쪽 → 오른쪽 (가로)	위 → 아래 (세로)
<code>row-reverse</code>	오른쪽 → 왼쪽	위 → 아래
<code>column</code>	위 → 아래 (세로)	왼쪽 → 오른쪽
<code>column-reverse</code>	아래 → 위	왼쪽 → 오른쪽

주의: `justify-content` 는 항상 **주축** 방향, `align-items` 는 항상 **교차축** 방향으로 정렬합니다. `flex-direction: column` 이면 `justify-content` 가 세로 정렬을 담당합니다. 방향이 헷갈리면 "justify = 주축, align = 교차축"으로 기억하세요.

justify-content와 align-items

`justify-content` 는 아이টে을 주축 방향으로 어떻게 분배할지 결정합니다.

값	의미
<code>flex-start</code> (기본값)	주축 시작점에 모음
<code>flex-end</code>	주축 끝점에 모음
<code>center</code>	주축 가운데 모음
<code>space-between</code>	양 끝에 붙이고 나머지 간격 균등 분배
<code>space-around</code>	각 아이টে 양쪽에 균등 간격
<code>space-evenly</code>	모든 간격(양 끝 포함) 균등 분배

`align-items` 는 아이টে을 교차축 방향으로 정렬합니다.

값	의미
<code>stretch</code> (기본값)	교차축 방향으로 컨테이너에 맞게 늘어남
<code>flex-start</code>	교차축 시작점에 붙임
<code>flex-end</code>	교차축 끝점에 붙임
<code>center</code>	교차축 가운데 정렬
<code>baseline</code>	텍스트 기준선 맞춤

코드로 확인하기 — 기본 Flexbox 정렬

다음 코드는 Flexbox의 대표적인 정렬 패턴들을 한 페이지에서 비교합니다.

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>Flexbox 정렬 비교</title>
  <style>
    body {
      font-family: sans-serif;
      padding: 16px;
    }

    .section-title {
      font-weight: bold;
      margin: 20px 0 6px;
      color: #444;
    }

    /* 공통 컨테이너 스타일 */
    .flex-container {
      display: flex;
      background-color: #f0f4ff;
      border: 2px solid #6c8ebf;
      border-radius: 8px;
      height: 80px;
      padding: 8px;
      gap: 8px;
    }

    /* 아이템 공통 스타일 */
    .flex-item {
      background-color: #2c7be5;
      color: #fff;
      border-radius: 6px;
      padding: 8px 16px;
      font-size: 14px;
      white-space: nowrap;
    }

    /* 정렬 변형 */
    .justify-center { justify-content: center; }
    .justify-between { justify-content: space-between; }
    .justify-evenly { justify-content: space-around; }
    .align-center { align-items: center; }
    .align-flex-end { align-items: flex-end; }

    /* 세로 방향 컨테이너 */
    .flex-column {
      flex-direction: column;
      height: 160px;
    }
  </style>
</head>
```

```

</style>
</head>
<body>
  <p class="section-title">기본값 (flex-start, stretch)</p>
  <div class="flex-container">
    <div class="flex-item">A</div>
    <div class="flex-item">B</div>
    <div class="flex-item">C</div>
  </div>

  <p class="section-title">justify-content: center + align-items: center</p>
  <div class="flex-container justify-center align-center">
    <div class="flex-item">A</div>
    <div class="flex-item">B</div>
    <div class="flex-item">C</div>
  </div>

  <p class="section-title">justify-content: space-between + align-items: center</p>
  <div class="flex-container justify-between align-center">
    <div class="flex-item">왼쪽</div>
    <div class="flex-item">가운데</div>
    <div class="flex-item">오른쪽</div>
  </div>

  <p class="section-title">justify-content: space-around + align-items: center</p>
  <div class="flex-container justify-around align-center">
    <div class="flex-item">A</div>
    <div class="flex-item">B</div>
    <div class="flex-item">C</div>
  </div>

  <p class="section-title">flex-direction: column (세로 정렬)</p>
  <div class="flex-container flex-column align-center">
    <div class="flex-item">위</div>
    <div class="flex-item">중간</div>
    <div class="flex-item">아래</div>
  </div>
</body>
</html>

```

브라우저 렌더링 결과:

기본값 (flex-start, stretch)

[A가득] [B가득] [C가득] ← 좌측 상단에 붙고 높이는 컨테이너에 맞게 늘어남

justify-content: center + align-items: center

[A] [B] [C] ← 가로·세로 모두 가운데

justify-content: space-between + align-items: center

[왼쪽] [가운데] [오른쪽] ← 양 끝 붙이고 나머지 균등

justify-content: space-evenly + align-items: flex-end

[A] [B] [C] ← 모든 간격 균등, 아이템은 아래에 붙음

flex-direction: column (세로 정렬)

[위]

[중간] ← 세로로 나열, 가로 가운데

[아래]

줄별 코드 설명

줄	코드	설명
17	<code>display: flex;</code>	이 요소를 플렉스 컨테이너로 만듭니다. 자식들이 플렉스 아이템이 됩니다
22	<code>gap: 8px;</code>	플렉스 아이템 사이의 간격을 8px로 설정합니다
33	<code>.justify-center { justify-cont...</code>	아이템들을 주축(가로) 가운데로 모읍니다
34	<code>.justify-between { justify-con...</code>	첫·마지막은 양 끝에, 나머지 간격을 균등 분배합니다
36	<code>.align-center { align-items: c...</code>	아이템들을 교차축(세로) 가운데로 정렬합니다
40	<code>flex-direction: column;</code>	주축을 세로로 바꿉니다. 아이템이 위에서 아래로 쌓입니다

flex-grow, flex-shrink, flex-basis — 크기 분배

아이템의 크기 분배를 제어하는 세 속성입니다.

- **flex-grow**: 남은 공간을 얼마나 차지할지 비율 (기본값 `0` → 늘어나지 않음)
- **flex-shrink**: 공간이 부족할 때 얼마나 줄어들지 비율 (기본값 `1` → 균등하게 줄어듦)

- **flex-basis** : 아이템의 기본 크기 (기본값 **auto** → 내용 크기)

세 속성을 한 번에 설정하는 **flex** 단축 속성을 사용하는 것이 권장됩니다.

코드로 확인하기 — flex-grow로 공간 분배

다음 코드는 **flex-grow** 로 남은 공간을 비율에 따라 나누는 방법을 보여줍니다.

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>flex-grow 비율 분배</title>
  <style>
    .container {
      display: flex;
      gap: 8px;
      background-color: #f0f4ff;
      border: 2px solid #6c8ebf;
      padding: 8px;
      border-radius: 8px;
    }

    .item {
      background-color: #2c7be5;
      color: #fff;
      padding: 12px;
      border-radius: 6px;
      text-align: center;
      font-size: 13px;
    }

    /* flex: grow shrink basis 단축 표기 */
    .item-1 { flex: 1; } /* 남은 공간을 1 비율로 차지 */
    .item-2 { flex: 2; } /* 남은 공간을 2 비율로 차지 */
    .item-3 { flex: 1; } /* 남은 공간을 1 비율로 차지 */
  </style>
</head>
<body>
  <div class="container">
    <div class="item item-1">flex: 1<br>(25%)</div>
    <div class="item item-2">flex: 2<br>(50%)</div>
    <div class="item item-3">flex: 1<br>(25%)</div>
  </div>
</body>
</html>
```

브라우저 렌더링 결과:

[—flex:1—][——flex:2——][—flex:1—]
(25%) (50%) (25%)
전체 가로 공간이 1:2:1 비율로 분배됨

줄별 코드 설명

줄	코드	설명
25	<code>flex: 1;</code>	<code>flex-grow: 1; flex-shrink: 1; flex-basis: 0;</code> 의 단축 표기입니다
26	<code>flex: 2;</code>	이 아이템은 <code>flex: 1</code> 아이템보다 두 배 넓게 공간을 차지합니다

`display: flex` - Flexbox 컨테이너 속성 요약

항목	내용
<code>flex-direction</code>	<code>row</code> (기본) <code>row-reverse</code> <code>column</code> <code>column-reverse</code> — 주축 방향
<code>justify-content</code>	<code>flex-start</code> (기본) <code>flex-end</code> <code>center</code> <code>space-between</code> <code>space-around</code> <code>space-evenly</code> — 주축 정렬
<code>align-items</code>	<code>stretch</code> (기본) <code>flex-start</code> <code>flex-end</code> <code>center</code> <code>baseline</code> — 교차축 정렬
<code>flex-wrap</code>	<code>nowrap</code> (기본) <code>wrap</code> — 줄바꿈 여부
<code>gap</code>	<code><길이></code> — 아이템 간 간격
<code>align-content</code>	여러 줄일 때 교차축 전체 정렬 (<code>flex-wrap: wrap</code> 일 때 사용)

`flex` - 플렉스 아이템 크기 단축 속성

항목	내용
설명	<code>flex-grow</code> , <code>flex-shrink</code> , <code>flex-basis</code> 를 한 번에 설정합니다
문법	<code>flex: <grow> <shrink> <basis></code>
기본값	<code>flex: 0 1 auto</code>
자주 쓰는 값	<code>flex: 1</code> → <code>flex: 1 1 0</code> (남은 공간 균등 분배)
사용 예시	<code>flex: 2</code> → <code>flex-grow</code> 비율을 2로 지정

팁: 요소를 수평·수직 완전 가운데 정렬하려면 컨테이너에 `display: flex; justify-content: center; align-items: center;` 세 줄이면 충분합니다. 과거에는 이것이 매우 복잡했지만, Flexbox 덕분에 간단해졌습니다.

절 요약

- `display: flex` 로 플렉스 컨테이너를 만들면 자식이 자동으로 플렉스 아이템이 됩니다
- `flex-direction` 으로 주축 방향을 결정합니다 (`row` = 가로, `column` = 세로)
- `justify-content` 는 주축 정렬, `align-items` 는 교차축 정렬입니다
- `flex: 1` 처럼 단축 속성으로 남은 공간 비율을 나눌 수 있습니다
- `gap` 으로 아이템 간 간격을 한 번에 지정합니다

6.3 Grid로 2차원 레이아웃 만들기

도입

Flexbox가 "한 줄의 선반"이라면, **Grid(그리드)**는 "격자 무늬의 모눈종이"입니다. 행(row)과 열(column)을 동시에 제어해 페이지 전체 레이아웃처럼 2차원으로 요소를 배치할 때 사용합니다. 헤더·사이드바·본문·푸터가 있는 전형적인 웹 페이지 레이아웃은 Grid로 몇 줄의 CSS만으로 완성할 수 있습니다.

핵심 개념 설명

그리드 컨테이너와 트랙(Track)

`display: grid`를 선언한 요소가 **그리드 컨테이너(grid container)**가 됩니다. 컨테이너 안에 정의된 행과 열을 **트랙(track)**이라고 하며, 행과 열이 교차하여 만들어지는 각 칸을 **셀(cell)**이라고 합니다.

트랙과 셀 사이의 **경계선**을 **그리드 라인(grid line)**이라고 합니다. **라인**은 1번부터 번호가 매겨집니다. 열이 3개라면 열 라인은 1, 2, 3, 4번이 됩니다.

grid-template-columns와 grid-template-rows

열과 행의 개수와 크기를 정의합니다.

```
.container {  
  display: grid;  
  grid-template-columns: 200px 1fr 1fr; /* 열: 200px 고정, 나머지 1:1 비율 */  
  grid-template-rows: 80px auto 60px; /* 행: 첫 행 80px, 두 번째 자동, 세 번째 */  
}
```

fr 단위와 repeat() 함수

fr (fraction)은 남은 공간의 비율을 나타내는 Grid 전용 단위입니다. Flexbox의 `flex: 1`과 비슷한 개념입니다.

`repeat()` 함수는 같은 패턴을 반복할 때 사용합니다.

```
/* 1fr씩 3열 - repeat 없이 */  
grid-template-columns: 1fr 1fr 1fr;  
  
/* repeat()로 동일하게 */  
grid-template-columns: repeat(3, 1fr);  
  
/* 250px 고정 열 + 나머지 균등 2열 */  
grid-template-columns: 250px repeat(2, 1fr);
```

gap으로 간격 주기

`gap` 속성으로 행과 열 사이 간격을 설정합니다. `row-gap` 과 `column-gap` 을 각각 지정할 수도 있습니다.

```
gap: 16px;           /* 행·열 간격 동일 */
gap: 20px 12px;      /* 행 간격 20px, 열 간격 12px */
```

CSS Grid 2차원 구조: 행/열 트랙과 `grid-area` 배치 예시

grid-area로 영역 이름 배치하기

`grid-template-areas` 와 `grid-area` 를 조합하면 레이아웃을 ASCII 아트처럼 직관적으로 그릴 수 있습니다.

코드로 확인하기 — 기본 Grid 격자

다음 코드는 3열 그리드로 카드들을 균등하게 배치합니다.

```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>Grid 3열 배치</title>
  <style>
    .grid-container {
      display: grid;                                /* Grid 활성화 */
      grid-template-columns: repeat(3, 1fr);        /* 균등한 3열 */
      gap: 16px;                                    /* 셀 간 간격 */
      padding: 16px;
      background-color: #f0f4ff;
      border: 2px solid #6c8ebf;
      border-radius: 8px;
    }

    .grid-item {
      background-color: #2c7be5;
      color: #fff;
      padding: 24px 16px;
      border-radius: 8px;
      text-align: center;
      font-weight: bold;
    }

    /* 특정 아이템이 2열 차지 */
    .span-2 {
      grid-column: span 2; /* 이 아이템은 열 2칸을 차지 */
      background-color: #1558b0;
    }
  </style>
</head>
<body>
  <div class="grid-container">
    <div class="grid-item">1</div>
    <div class="grid-item">2</div>
    <div class="grid-item">3</div>
    <div class="grid-item span-2">4 (2열 차지)</div>
    <div class="grid-item">5</div>
    <div class="grid-item">6</div>
    <div class="grid-item">7</div>
    <div class="grid-item">8</div>
  </div>
</body>
</html>

```

브라우저 렌더링 결과:

```
[ 1 ] [ 2 ] [ 3 ]  
[ 4 (2열 차지) ] [ 5 ]  
[ 6 ] [ 7 ] [ 8 ]
```

줄별 코드 설명

줄	코드	설명
7	<code>display: grid;</code>	이 요소를 그리드 컨테이너로 만듭니다
8	<code>grid-template-columns: repeat(...</code>	3개의 균등한 열을 만듭니다. <code>repeat(3, 1fr)</code> 은 <code>1fr 1fr 1fr</code> 과 동일합니다
9	<code>gap: 16px;</code>	행과 열 사이 간격을 16px로 설정합니다
31	<code>grid-column: span 2;</code>	이 아이템이 열 2칸을 차지하도록 확장합니다

코드로 확인하기 — grid-area로 페이지 레이아웃

다음 코드는 헤더·사이드바·본문·푸터를 `grid-template-areas` 로 배치하는 전형적인 페이지 레이아웃입니다.


```
</body>
</html>
```

브라우저 렌더링 결과:

```
[————— 헤더 (header) —————]
[사이드바 ][————— 본문 (main) —————]
[(sidebar) ][
[————— 푸터 (footer) —————]
```

줄별 코드 설명

줄	코드	설명
7	<code>grid-template-columns: 200px 1fr</code>	첫 번째 열은 200px 고정, 두 번째 열은 나머지 전체입니다
8	<code>grid-template-rows: 60px 1fr 50px</code>	행 높이를 순서대로 정의합니다
9-12	<code>grid-template-areas: ...</code>	영역 이름을 격자 형태의 문자열로 배치합니다. 같은 이름을 연속으로 쓰면 그 영역이 해당 칸들을 모두 차지합니다
16	<code>grid-area: header;</code>	이 요소가 <code>grid-template-areas</code> 에서 "header"로 정의된 영역을 차지합니다

display: grid - Grid 컨테이너 속성 요약

항목	내용
grid-template-columns	열 트랙 정의. 예: <code>repeat(3, 1fr)</code> , <code>200px 1fr 1fr</code>
grid-template-rows	행 트랙 정의. 예: <code>80px auto 60px</code>
grid-template-areas	영역 이름으로 격자 레이아웃 정의
gap	행·열 간격. <code>row-gap</code> / <code>column-gap</code> 으로 개별 설정 가능
grid-area	아이템 속성. <code>grid-template-areas</code> 에 정의한 이름을 지정
grid-column	아이템 속성. 열 라인 번호로 위치 지정. 예: <code>1 / 3</code> , <code>span 2</code>
grid-row	아이템 속성. 행 라인 번호로 위치 지정

팁: `grid-template-areas` 를 사용하면 HTML 구조를 바꾸지 않고도 CSS만으로 레이아웃을 완전히 재배치할 수 있습니다. 반응형 디자인(7장)에서 미디어 쿼리와 함께 쓰면 화면 크기에 따라 레이아웃을 쉽게 변경할 수 있습니다.

절 요약

- `display: grid` 로 2차원 격자 레이아웃을 만듭니다
- `grid-template-columns` 와 `grid-template-rows` 로 트랙 크기를 정의합니다
- `fr` 단위는 남은 공간을 비율로 나눕니다. `repeat()` 으로 반복 패턴을 간결하게 씁니다
- `grid-template-areas` + `grid-area` 로 영역 이름 기반의 직관적인 배치가 가능합니다
- `gap` 으로 셀 간 간격을 한 번에 지정합니다

6.4 포지셔닝과 z-index

도입

Flexbox와 Grid가 여러 요소의 전체적인 배치를 다룬다면, `position` 속성은 특정 요소 하나를 정상 흐름에서 빼내어 원하는 위치에 정확히 고정하는 데 사용합니다. 화면 오른쪽 아래에 고정된 "채팅 버튼", 카드 모서리에 붙은 "NEW 배지", 스크롤해도 따라오는 "상단 네비게이션"이 모두 `position` 으로 구현됩니다.

핵심 개념 설명

position 값 비교

값	정상 흐름	기준	주요 용도
<code>static</code> (기본 값)	유지	없음	기본 상태, <code>top</code> / <code>left</code> 무효
<code>relative</code>	유지	자신의 원래 위치	오프셋으로 살짝 이동, <code>absolute</code> 의 기준 컨테이너 역할
<code>absolute</code>	이탈	가장 가까운 <code>position</code> 지정 조상	특정 위치에 정밀 배치
<code>fixed</code>	이탈	뷰포트(브라우저 화면)	스크롤에 고정되는 헤더·버튼
<code>sticky</code>	유지 → 이탈	가장 가까운 스크롤 컨테이너	스크롤 시 상단 고정

CSS position 속성 값별 요소 배치 비교 (static/relative/absolute/fixed/sticky)

relative와 absolute의 관계

`position: absolute` 는 가장 가까운 `position` 지정 조상(`position: relative/absolute/fixed/sticky`)을 기준으로 배치됩니다. 그런 조상이 없으면 `<html>` 루트를 기준으로 합니다.

이 특성을 이용해 카드 모서리에 배지를 붙이는 패턴이 자주 사용됩니다. 부모 카드에 `position: relative` 를 주고, 배지 요소에 `position: absolute` 와 `top` , `right` 오프셋을 지정합니다.

주의: `position: absolute` 가 부모가 아닌 조상 기준으로 배치된다고 오해하기 쉽습니다. 정확히는 `position: static` 이 아닌 **가장 가까운 조상**이 기준입니다. 따라서 의도한 곳을 기준으로 삼으려면 해당 부모에 `position: relative` 를 반드시 지정해야 합니다.

z-index와 쌓임 맥락(Stacking Context)

요소가 겹칠 때 앞뒤 순서를 `z-index` 로 제어합니다. 숫자가 클수록 앞에 표시됩니다. `z-index` 는 `position: static` 이 아닌 요소에만 작동합니다.

쌓임 맥락(Stacking Context)은 새로운 z-index 기준점입니다. `position` 이 `static` 이외의 값이고 `z-index` 가 `auto` 가 아닌 요소는 새로운 쌓임 맥락을 만듭니다. 쌓임 맥락 안의 `z-index` 는 그 맥락 내부에서만 비교됩니다.

코드로 확인하기 — `relative`와 `absolute`로 배지 붙이기

다음 코드는 카드 오른쪽 위 모서리에 "NEW" 배지를 `position: absolute` 로 고정합니다.

```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>포지셔닝 실습 - 배지</title>
  <style>
    .card-container {
      display: flex;
      gap: 20px;
      padding: 20px;
    }

    /* 카드: relative로 자식의 absolute 기준 역할 */
    .card {
      position: relative;          /* absolute 자식의 기준점 */
      width: 180px;
      padding: 20px 16px;
      background-color: #f8f9fa;
      border: 1px solid #dee2e6;
      border-radius: 12px;
      box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);
    }

    .card-title {
      font-weight: bold;
      margin-bottom: 8px;
      color: #333;
    }

    .card-desc {
      font-size: 13px;
      color: #666;
    }

    /* 배지: absolute로 카드 오른쪽 위 모서리에 고정 */
    .badge {
      position: absolute;          /* 부모 .card를 기준으로 배치 */
      top: -10px;                 /* 카드 상단에서 위로 10px */
      right: -10px;               /* 카드 우측에서 바깥으로 10px */
      background-color: #e63946;
      color: #fff;
      font-size: 11px;
      font-weight: bold;
      padding: 4px 8px;
      border-radius: 20px;
    }

    /* fixed 버튼: 화면 오른쪽 아래에 고정 */
    .floating-btn {
      position: fixed;            /* 뷰포트 기준 고정 */

```

```

    bottom: 20px;
    right: 20px;
    background-color: #2c7be5;
    color: #fff;
    border: none;
    border-radius: 50px;
    padding: 12px 20px;
    font-size: 14px;
    cursor: pointer;
    box-shadow: 0 4px 12px rgba(44, 123, 229, 0.4);
  }
</style>
</head>
<body>
  <div class="card-container">
    <div class="card">
      <span class="badge">NEW</span>      <!-- 카드 모서리에 배지 -->
      <p class="card-title">상품 이름 A</p>
      <p class="card-desc">설명 텍스트가 들어갑니다.</p>
    </div>

    <div class="card">
      <span class="badge">SALE</span>
      <p class="card-title">상품 이름 B</p>
      <p class="card-desc">설명 텍스트가 들어갑니다.</p>
    </div>

    <div class="card">
      <!-- 배지 없음 -->
      <p class="card-title">상품 이름 C</p>
      <p class="card-desc">설명 텍스트가 들어갑니다.</p>
    </div>
  </div>

  <button class="floating-btn">+ 새로 추가</button>
</body>
</html>

```

브라우저 렌더링 결과:

[NEW] [SALE]

[상품 이름 A] [상품 이름 B] [상품 이름 C]

[설명 텍스트...] [설명 텍스트...] [설명 텍스트...]

(화면 오른쪽 하단에 고정)

[+ 새로 추가]

줄별 코드 설명

줄	코드	설명
13	<code>position: relative;</code>	이 카드를 자식 <code>absolute</code> 요소의 기준 컨테이너로 만듭니다. 카드 자체의 위치는 바뀌지 않습니다
30	<code>position: absolute;</code>	정상 흐름에서 이탈하여 가장 가까운 position 지정 조상(<code>.card</code>)을 기준으로 배치됩니다
31-32	<code>top: -10px; right: -10px;</code>	카드의 위 경계에서 위로 10px, 오른쪽 경계에서 바깥으로 10px 위치에 배치를 놓습니다
43	<code>position: fixed;</code>	뷰포트(브라우저 화면 전체)를 기준으로 배치됩니다. 스크롤해도 위치가 고정됩니다
44-45	<code>bottom: 20px; right: 20px;</code>	화면 오른쪽 아래에서 각각 20px 떨어진 위치에 고정합니다

코드로 확인하기 — z-index로 겹침 순서 제어

다음 코드는 `z-index` 로 요소의 앞뒤를 제어합니다.

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>z-index 겹침 제어</title>
  <style>
    .stack-container {
      position: relative;    /* 자식의 absolute 기준 */
      width: 300px;
      height: 200px;
      border: 1px solid #ccc;
    }

    .box {
      position: absolute;    /* z-index 적용을 위해 position 필요 */
      width: 150px;
      height: 100px;
      border-radius: 8px;
      display: flex;
      align-items: center;
      justify-content: center;
      font-weight: bold;
      color: #fff;
    }

    .box-a {
      background-color: rgba(44, 123, 229, 0.85);
      top: 20px;
      left: 20px;
      z-index: 1;            /* 가장 뒤 */
    }

    .box-b {
      background-color: rgba(230, 57, 70, 0.85);
      top: 60px;
      left: 80px;
      z-index: 2;            /* 가운데 */
    }

    .box-c {
      background-color: rgba(38, 166, 91, 0.85);
      top: 100px;
      left: 140px;
      z-index: 3;            /* 가장 앞 */
    }
  </style>
</head>
<body>
  <div class="stack-container">
    <div class="box box-a">파랑 (z:1)</div>
```

```

    <div class="box box-b">빨강 (z:2)</div>
    <div class="box box-c">초록 (z:3)</div>
  </div>
</body>
</html>

```

브라우저 렌더링 결과:

[파랑(z:1)]
 [빨강(z:2)] ← 파랑 위에 겹침
 [초록(z:3)] ← 빨강 위에 겹침
 (숫자가 클수록 앞에 표시)

줄별 코드 설명

줄	코드	설명
11	<code>position: absolute;</code>	<code>z-index</code> 를 사용하려면 <code>position: static</code> 이외의 값이 필요합니다
27	<code>z-index: 1;</code>	쌓임 순서의 가장 낮은 값. 다른 두 박스 뒤에 위치합니다
34	<code>z-index: 2;</code>	중간 순서. 파랑 박스 앞, 초록 박스 뒤에 위치합니다
41	<code>z-index: 3;</code>	가장 높은 순서. 모든 박스 앞에 표시됩니다

팁: `position: sticky` 는 처음에는 정상 흐름에 있다가 스크롤 시 지정한 위치(`top: 0` 등)에 도달하면 고정됩니다. 섹션 제목이 스크롤 시 상단에 붙는 효과를 구현할 때 유용합니다. 부모 요소를 벗어나면 고정이 풀립니다.

절 요약

- `position: relative` 는 원래 위치를 유지하며 오프셋으로 이동. 자식 `absolute` 의 기준 컨테이너 역할
- `position: absolute` 는 가장 가까운 `position` 지정 조상을 기준으로 배치, 정상 흐름에서 이탈

- `position: fixed` 는 뷰포트 기준 고정. 스크롤에 영향을 받지 않음
- `position: sticky` 는 스크롤 시 지정 위치에서 고정으로 전환
- `z-index` 는 `position: static` 이외의 요소에서만 작동하며, 숫자가 클수록 앞에 표시됨

6.5 Flexbox와 Grid 언제 무엇을 쓸까

도입

Flexbox와 Grid를 모두 배우고 나면 "어떤 상황에서 무엇을 써야 하는가?"가 자연스럽게 떠오릅니다. 둘 다 레이아웃을 위한 도구이지만, 각자가 잘하는 영역이 다릅니다. 선택 기준을 명확히 하면 코드가 간결해지고 의도가 분명해집니다.

핵심 개념 설명

1차원 vs 2차원

가장 핵심적인 기준은 **배치 방향**입니다.

	Flexbox	Grid
차원	1차원 (한 방향)	2차원 (행 + 열)
콘텐츠 주도	아이템 크기에 맞게 컨테이너가 조절	컨테이너 틀에 맞게 아이템이 배치
주요 사용처	네비게이션 바, 버튼 그룹, 카드 내부 구성	페이지 전체 레이아웃, 갤러리 격자, 대시보드

Flexbox가 적합한 경우

- 가로 또는 세로 **한 방향**으로만 정렬할 때
- 아이템 개수가 동적으로 변하고 콘텐츠 크기에 맞게 유연하게 늘어나야 할 때
- 버튼 그룹, 입력 필드와 버튼의 가로 배치, 아이콘 + 텍스트 정렬
- 카드 내부(이미지 + 제목 + 설명 + 버튼)처럼 단일 방향 흐름

Grid가 적합한 경우

- 행과 열을 **동시에** 제어해야 할 때
- 페이지 전체 레이아웃(헤더, 사이드바, 본문, 푸터)
- 이미지 갤러리, 대시보드처럼 격자 형태가 필요할 때
- 아이টে을 특정 셀에 정확히 배치해야 할 때

둘을 함께 중첩하여 사용하기

실제 프로젝트에서는 두 가지를 함께 사용하는 것이 일반적입니다. **바깥 컨테이너는 Grid로 전체 틀을 잡고, 안쪽 요소는 Flexbox로 세부 정렬을 합니다.**

코드로 확인하기 — Grid + Flexbox 중첩 레이아웃

다음 코드는 Grid로 전체 갤러리 격자를 만들고, Flexbox로 각 카드 내부를 정렬합니다.

```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>Grid + Flexbox 중첩</title>
  <style>
    /* Grid: 전체 카드 갤러리 격자 */
    .gallery {
      display: grid;
      grid-template-columns: repeat(3, 1fr);
      gap: 20px;
      padding: 20px;
      background-color: #f8f9fa;
    }

    /* Flexbox: 카드 내부 구성 요소 정렬 */
    .card {
      display: flex;
      flex-direction: column;
      background-color: #fff;
      border: 1px solid #e0e0e0;
      border-radius: 12px;
      overflow: hidden;
      box-shadow: 0 2px 8px rgba(0,0,0,0.08);
    }

    .card-image {
      width: 100%;
      height: 140px;
      background-color: #a8d8ea;
      display: flex;
      align-items: center;
      justify-content: center;
      font-size: 32px;
    }

    .card-body {
      padding: 16px;
      flex: 1;
      display: flex;
      flex-direction: column;
      gap: 8px;
    }

    .card-title {
      font-weight: bold;
      font-size: 16px;
      color: #222;
    }
  </style>

```

```

    .card-desc {
      font-size: 13px;
      color: #666;
      flex: 1;
    }

    .card-btn {
      display: block;
      background-color: #2c7be5;
      color: #fff;
      text-align: center;
      padding: 8px;
      border-radius: 6px;
      text-decoration: none;
      font-size: 14px;
      margin-top: auto;
    }
  </style>
</head>
<body>
  <div class="gallery">
    <div class="card">
      <div class="card-image">🖼️</div>
      <div class="card-body">
        <p class="card-title">카드 제목 A</p>
        <p class="card-desc">짧은 설명</p>
        <a href="#" class="card-btn">자세히 보기</a>
      </div>
    </div>

    <div class="card">
      <div class="card-image">🖼️</div>
      <div class="card-body">
        <p class="card-title">카드 제목 B</p>
        <p class="card-desc">이 카드는 설명 텍스트가 조금 더 길어서 두 줄 이상 차지<
        <a href="#" class="card-btn">자세히 보기</a>
      </div>
    </div>

    <div class="card">
      <div class="card-image">🖼️</div>
      <div class="card-body">
        <p class="card-title">카드 제목 C</p>
        <p class="card-desc">짧은 설명</p>
        <a href="#" class="card-btn">자세히 보기</a>
      </div>
    </div>
  </div>
</body>
</html>

```

브라우저 렌더링 결과:

[이미지 A] [이미지 B] [이미지 C]
[카드 제목 A] [카드 제목 B] [카드 제목 C]
[짧은 설명] [긴 설명] [짧은 설명]
[] [텍스트가] []
[자세히 보기] [자세히 보기] [자세히 보기]
(각 카드 높이가 가장 큰 카드에 맞춰 정렬됨, 버튼은 항상 하단에 위치)

줄별 코드 설명

줄	코드	설명
8	<code>display: grid;</code>	갤러리 전체 틀을 그리드로 만듭니다
9	<code>grid-template-columns: repeat(...</code>	3열 균등 격자를 정의합니다
16	<code>display: flex; flex-direction:...</code>	카드 내부를 세로 플렉스 컨테이너로 만듭니다
35	<code>flex: 1;</code>	카드 바디가 남은 공간을 채워 카드 높이를 균일하게 만듭니다
47	<code>flex: 1;</code>	설명 텍스트가 공간을 차지해 버튼을 자연스럽게 아래로 밀어냅니다
55	<code>margin-top: auto;</code>	앞 요소와의 간격을 최대화해 버튼을 항상 카드 맨 아래에 고정합니다

비교 테이블 — Flexbox vs Grid 선택 가이드

기준	Flexbox	Grid
배치 방향	1차원 (가로 또는 세로)	2차원 (행과 열 동시)
콘텐츠 기준	아이템 크기에 맞게 유연	미리 정의된 격자에 배치
주요 사용	네비게이션, 버튼 그룹, 카드 내부	페이지 레이아웃, 갤러리, 대시보드
브라우저 지원	모든 최신 브라우저 완전 지원	모든 최신 브라우저 완전 지원
반응형	<code>flex-wrap: wrap</code> 으로 줄바꿈	<code>auto-fit</code> / 미디어 쿼리 조합

베스트 프랙티스: Flexbox와 Grid는 경쟁 관계가 아닙니다. 페이지 전체 골격은 Grid, 개별 컴포넌트 내부는 Flexbox를 사용하는 것이 자연스러운 조합입니다.

절 요약

- Flexbox: 1차원(단일 방향) 정렬, 콘텐츠 기반 크기
- Grid: 2차원(행+열) 배치, 컨테이너 기반 배치
- 실무에서는 Grid(전체 틀) + Flexbox(컴포넌트 내부)로 중첩 사용이 일반적

FAQ

Q1: `justify-content` 와 `align-items` 가 항상 헛갈립니다. 쉽게 기억하는 방법이 있나요?

A1: "justify = 주축, align = 교차축"으로 기억하세요. 주축 방향은 `flex-direction` 이 결정합니다. `flex-direction: row` (기본값)이면 주축이 가로이므로 `justify-content` 가 가로 정렬, `align-items` 가 세로 정렬입니다. `flex-direction: column` 이면 반대가 됩니다.

Q2: `position: absolute` 요소가 원하는 곳이 아닌 엉뚱한 위치에 배치됩니다. 왜 그럴까요?

A2: `position: absolute` 는 `position: static` 이 아닌 가장 가까운 조상을 기준으로 삼습니다. 원하는 부모 요소에 `position: relative` 가 없으면 더 바깥쪽 조상(또는 `<html>`)이 기준이 됩니다. 의도한 부모에 `position: relative;` 를 추가하면 해결됩니다.

Q3: Grid와 Flexbox 중 어떤 것을 먼저 배워야 하나요?

A3: 일반적으로 Flexbox를 먼저 배우는 것이 좋습니다. 개념이 더 단순하고, 컴포넌트 단위 정렬에서 빈번히 사용됩니다. Flexbox에 익숙해진 후 Grid를 배우면 "1차원은 Flexbox, 2차원은 Grid"라는 차이가 명확하게 느껴집니다.

Q4: `fr` 단위는 `%` 단위와 무엇이 다른가요?

A4: `%` 는 부모 요소의 크기 기준으로 계산되며 `gap` 을 포함한 전체 크기의 비율입니다. `fr` 은 `gap` 을 제외한 남은 공간을 비율로 나눕니다. 예를 들어 `gap: 20px` 이 있는 3열 레이아웃에서 각 열을 `1fr` 로 지정하면 `gap` 공간을 자동으로 빼고 남은 공간을 균등 분배합니다. 반면 `33.33%` 로 지정하면 `gap` 크기만큼 가로를 벗어날 수 있습니다.

Q5: `z-index` 를 높게 설정했는데 앞에 오지 않습니다. 왜 그럴까요?

A5: `z-index` 는 두 가지 조건이 있을 때만 작동합니다. 첫째, `position: static` 이외의 값 (`relative`, `absolute`, `fixed`, `sticky`)이어야 합니다. 둘째, 같은 **쌓임 맥락(stacking context)** 안에서만 비교됩니다. 부모 요소가 새로운 쌓임 맥락을 만들면 그 내부의 `z-index` 는 부모를 벗어날 수 없습니다.

장 요약

이번 장에서는 CSS 레이아웃의 핵심 도구들을 학습했습니다. `display` 속성으로 요소의 기본 배치 방식(블록, 인라인, 인라인 블록)을 이해하고, Flexbox로 1차원 정렬(주축/교차축, justify-content, align-items)을, Grid로 2차원 격자 배치(`grid-template-columns`, `fr` 단위, `grid-area`)를 구현하는 방법을 배웠습니다. 또한 `position` 속성으로 특정 요소를 정밀 배치하고 `z-index` 로 겹침 순서를 제어하는 방법도 살펴보았습니다. 마지막으로 "1차원이면 Flexbox, 2차원이면 Grid, 두 가지를 중첩해서 사용"이라는 실무 선택 기준을 익혔습니다.

핵심 키워드

`display`, `block`, `inline`, `inline-block`, Normal Flow, Flexbox, `flex-direction`, `justify-content`, `align-items`, `flex-grow`, Grid, `grid-template-columns`, `fr`, `repeat()`, `grid-area`, `gap`, `position`, `relative`, `absolute`, `fixed`, `sticky`, `z-index`, Stacking Context

다음 장 미리보기

다음 장에서는 이번 장에서 배운 Flexbox와 Grid를 활용해 다양한 화면 크기에 맞게 레이아웃을 자동으로 바꾸는 **반응형 웹 디자인**을 배웁니다. 미디어 쿼리와 모바일 우선 접근법으로 하나의 HTML 파일이 모바일·태블릿·데스크톱 모두에서 보기 좋은 페이지를 만드는 방법을 학습합니다.

✂ 실습 프로젝트 (Hands-on Project)

6장 실습 프로젝트: 카드 갤러리 레이아웃

이 장에서 배운 Flexbox, Grid, 포지셔닝 개념을 실무 시나리오에 적용하는 프로젝트입니다.

초급: 상품 카드 갤러리

시나리오

쇼핑몰 상품 목록 페이지를 만들어야 합니다. 상품 카드를 Grid로 격자 배치하고, 각 카드 내부는 Flexbox로 정렬합니다. 상품에는 "NEW", "SALE" 같은 배지가 카드 모서리에 고정되어야 합니다.

학습 목표

- `display: grid` 와 `grid-template-columns` 로 카드 격자를 만들 수 있습니다
- Flexbox(`flex-direction: column` , `margin-top: auto`)로 카드 내부를 정렬할 수 있습니다
- `position: relative + absolute` 로 배지를 카드 모서리에 고정할 수 있습니다

진행 방법

1. `project_starter.html` 을 열고 `TODO` 주석을 찾습니다
2. 각 TODO를 이 장에서 배운 Grid, Flexbox, position 개념으로 완성합니다
3. 브라우저에서 열어 카드 갤러리가 올바르게 표시되는지 확인합니다

실행 방법

브라우저에서 `project_starter.html` 파일을 직접 엽니다.
(VS Code Live Server 사용 가능)

완성 기준

- 카드가 3열 격자로 배치됩니다
- 각 카드의 "구매하기" 버튼이 항상 카드 하단에 위치합니다

- "NEW"/"SALE" 배지가 카드 오른쪽 위 모서리에 고정됩니다
- 모든 카드 높이가 같은 행 안에서 균일합니다

중급: 관리자 대시보드 레이아웃

시나리오

관리자 대시보드 페이지를 만들어야 합니다. Grid로 헤더·사이드바·본문·푸터 전체 레이아웃을 잡고, 본문 안의 통계 카드들은 Flexbox로 배치합니다. 사이드바 메뉴에는 현재 선택된 메뉴를 표시하는 active 상태가 있으며, 오른쪽 아래에는 고정 액션 버튼이 위치합니다.

학습 목표

- `grid-template-areas` 로 헤더·사이드바·본문·푸터 레이아웃을 구성할 수 있습니다
- Grid + Flexbox를 중첩하여 전체 레이아웃과 컴포넌트 내부를 모두 제어할 수 있습니다
- `position: fixed` 로 스크롤에 고정되는 버튼을 구현할 수 있습니다

진행 방법

1. `project_advanced_starter.html` 을 열고 `TODO` 주석을 찾습니다
2. 초급에서 연습한 내용을 바탕으로 더 복잡한 레이아웃을 완성합니다
3. 브라우저에서 열어 전체 대시보드 레이아웃을 확인합니다

실행 방법

브라우저에서 `project_advanced_starter.html` 파일을 직접 엽니다.

확장 아이디어

- 사이드바를 접고 펼치는 토글 기능을 JavaScript로 추가해 보세요
- 통계 카드를 클릭하면 상세 정보가 나타나는 기능을 추가해 보세요
- `position: sticky` 로 사이드바 헤더를 스크롤 시 고정해 보세요

▶  project_complete.html (완성 코드)

▶  project_advanced_starter.html (스타터 코드 - 직접 완성해보세요)

▶  project_advanced_complete.html (완성 코드)